The algorithm for pluralizations in `:en` is as simple as:

```
entry[count == 1 ? 0 : 1]
```

I.e. the translation denoted as `:one` is regarded as singular, the other is used as plural (including the count being zero).

If the lookup for the key does not return a Hash suitable for pluralization,

an `I18n::InvalidPluralizationData` exception is raised.

4.4 Setting and Passing a Locale

The locale can be either set pseudo-globally to `I18n.locale` (which uses `Thread.current` like, e.g.,`Time.zone`) or can be passed as an option to `#translate` and `#localize`.

If no locale is passed, `I18n.locale` is used:

```
I18n.locale = :de
I18n.t :foo
I18n.l Time.now
```

Explicitly passing a locale:

```
I18n.t :foo, locale: :de
I18n.l Time.now, locale: :de
```

The `I18n.locale` defaults to `I18n.default_locale` which defaults to :`en`. The default locale can be set like this:

```
I18n.default_locale = :de
```

4.5 Using Safe HTML Translations

Keys with a '_html' suffix and keys named 'html' are marked as HTML safe. When you use them in views the HTML will not be escaped.

```
# config/locales/en.yml
en:
  welcome: <b>welcome!</b>
  hello_html: <b>hello!</b>
  title:
    html: <b>title!</b>
# app/views/home/index.html.erb
<div><%= t('welcome') %></div>
<div><%= raw t('welcome') %></div>
<div><%= t('hello_html') %></div>
<div><%= t('title.html') %></div>
```

Interpolation escapes as needed though. For example, given:

```
en:
  welcome_html: "<b>Welcome %{username}!</b>"
```

you can safely pass the username as set by the user:

```
<%# This is safe, it is going to be escaped if needed. %>
<%= t('welcome_html', username: @current_user.username %>
```

Safe strings on the other hand are interpolated verbatim.

Automatic conversion to HTML safe translate text is only available from the `translate`view helper method.